Ciencia de Datos para Economía y Negocios - FCE-UBA

Clase 4 — Agrupamiento, pivots, condicionales y joins

Nicolás Sidicaro

Hoja de ruta de hoy

1. Parte I: group_by() + summarise() en profundidad
2. Parte II: Reestructuración de datos
3. Parte III: Condicionales
4. Parte IV: Modelo relacional
5. Parte V: Joins
6. Parte VI: Resumen y cierre

Parte I: group_by() + summarise() en profundidad

Repaso rápido

En la clase anterior vimos la combinación básica:

`group_by()` marca las categorías; `summarise()` calcula estadísticas **dentro de cada grupo**.

```
latam |>
  group_by(region) |>
  summarise(
    pbi_promedio = round(mean(pbi_pc), 0),
    n_obs = n()
  )
```

```
# A tibble: 3 x 3
  region       pbi_promedio n_obs
  <chr>               <dbl> <int>
1 Andina               6808     9
2 Cono Sur            14744     9
3 Norteamérica         9828     3
```

Agrupar por más de una variable

Podemos agrupar por **múltiples columnas** simultáneamente:

```
latam |>
  group_by(region, anio) |>
  summarise(
    pbi_promedio = round(mean(pbi_pc), 0),
    infl_promedio = round(mean(inflacion), 1),
    .groups = "drop"
  )
```

```
# A tibble: 9 x 4
  region        anio pbi_promedio infl_promedio
  <chr>        <int>        <dbl>         <dbl>
1 Andina        2020         6086           2.7
2 Andina        2021         6781           7.4
3 Andina        2022         7556           8.2
```

El argumento `.groups`

Cuando agrupamos por más de una variable, `summarise()` nos avisa cómo quedan los grupos residuales:

Valor	Comportamiento
`"drop_last"`	Elimina el **último** nivel de agrupamiento (default)
`"drop"`	Elimina **todos** los grupos
`"keep"`	Mantiene la misma estructura de grupos
`"rowwise"`	Cada fila queda como su propio grupo

Buena práctica

Siempre especificar `.groups = "drop"` para evitar sorpresas al encadenar operaciones después del `summarise()`.

group_by() + mutate(): operaciones dentro del grupo

group_by() no solo sirve con summarise(). Combinado con mutate(), permite calcular valores **relativos al grupo**:

```
latam |>
  filter(anio == 2022) |>
  group_by(region) |>
  mutate(
    pbi_max_region = max(pbi_pc),
    ratio_vs_max   = round(pbi_pc / pbi_max_region, 2)
  ) |>
  ungroup() |>
  select(pais, region, pbi_pc, ratio_vs_max)
```

```
# A tibble: 7 x 4
  pais  region       pbi_pc ratio_vs_max
  <chr> <chr>         <dbl>        <dbl>
```

group_by() + mutate(): participación en el total

```r
latam |>
  filter(anio == 2022) |>
  mutate(pbi_total = pbi_pc * poblacion_mill) |>
  group_by(region) |>
  mutate(
    pbi_total_region = sum(pbi_total),
    participacion    = round(pbi_total / pbi_total_region * 100, 1)
  ) |>
  ungroup() |>
  select(pais, region, pbi_total, participacion) |>
  arrange(region, desc(participacion))
```

```
# A tibble: 7 x 4
  pais  region       pbi_total participacion
  <chr> <chr>            <dbl>         <dbl>
1 BRA   Andina        1920045.          76.7
2 COL   Andina         343786.          13.7
3 PER   Andina         240146.           9.6
4 ARG   Cono Sur       630630          60.9
5 CHL   Cono Sur       329280          31.8
```

`group_by()` + `mutate()`: participación en el total

Diferencia clave

`summarise()` colapsa a una fila por grupo. `mutate()` conserva todas las filas y agrega la estadística grupal a cada una.

ungroup(): siempre desagrupar

Después de usar group_by() + mutate(), es importante **desagrupar**:

```
# Sin ungroup: las operaciones siguientes siguen agrupadas
datos_agrupados <- latam |>
  group_by(region) |>
  mutate(pbi_max = max(pbi_pc))
# ¡Cuidado! Si después hacés filter(), arrange(), etc.,
# operan DENTRO de cada grupo

# Con ungroup: volvemos al dataframe normal
datos_limpios <- latam |>
  group_by(region) |>
  mutate(pbi_max = max(pbi_pc)) |>
  ungroup()
```

`ungroup()`: siempre desagrupar

Regla clave

Si usás `group_by()` + `mutate()`, terminá siempre con `ungroup()`. Con `summarise()` no es necesario si usás `.groups = "drop"`.

Parte II: Reestructuración de datos

¿Por qué reestructurar?

Muchas veces los datos vienen en un formato que **no es el que necesitamos** para el análisis:

Formato ancho

- Una columna por período/categoría
- Cómodo para **leer** como tabla
- Típico de planillas Excel, INDEC

Formato largo (tidy)

- Una fila por observación
- Necesario para **dplyr y ggplot2**
- Facilita agrupamientos y filtros

Las funciones `pivot_longer()` y `pivot_wider()` (del paquete **tidyr**) permiten convertir entre ambos formatos.

Ejemplo: datos en formato ancho

```r
pbi_ancho <- tibble(
  pais = c("ARG", "BRA", "CHL", "COL", "MEX"),
  pbi_2020 = c(8571, 6797, 13231, 5334, 8347),
  pbi_2021 = c(10636, 7519, 16265, 6131, 10046),
  pbi_2022 = c(13650, 8918, 16800, 6624, 11091)
)
pbi_ancho
```

```
# A tibble: 5 x 4
  pais  pbi_2020 pbi_2021 pbi_2022
  <chr>    <dbl>    <dbl>    <dbl>
1 ARG       8571    10636    13650
2 BRA       6797     7519     8918
3 CHL      13231    16265    16800
4 COL       5334     6131     6624
5 MEX       8347    10046    11091
```

pivot_longer(): de ancho a largo

```r
pbi_largo <- pbi_ancho |>
  pivot_longer(
    cols      = pbi_2020:pbi_2022,
    names_to  = "anio",
    values_to = "pbi_pc"
  )
pbi_largo
```

```
# A tibble: 15 x 3
   pais  anio     pbi_pc
   <chr> <chr>     <dbl>
 1 ARG   pbi_2020   8571
 2 ARG   pbi_2021  10636
 3 ARG   pbi_2022  13650
 4 BRA   pbi_2020   6797
 5 BRA   pbi_2021   7519
```

pivot_longer(): limpiando nombres

El resultado tiene los nombres originales (pbi_2020, etc.) en la columna anio. Podemos limpiar esto:

```
pbi_largo <- pbi_ancho |>
  pivot_longer(
    cols         = starts_with("pbi_"),
    names_to     = "anio",
    names_prefix = "pbi_",
    values_to    = "pbi_pc"
  ) |>
  mutate(anio = as.integer(anio))

pbi_largo
```

```
# A tibble: 15 x 3
   pais   anio pbi_pc
```

`pivot_longer()`: limpiando nombres

`names_prefix` remueve el prefijo `"pbi_"` antes de guardar en `anio`. Luego convertimos a entero con `as.integer()`.

pivot_wider(): de largo a ancho

La operación inversa — útil para presentar tablas o hacer cálculos entre columnas:

```
pbi_largo |>
  pivot_wider(
    names_from  = anio,
    values_from = pbi_pc
  )
```

```
# A tibble: 5 x 4
  pais  `2020` `2021` `2022`
  <chr>  <dbl>  <dbl>  <dbl>
1 ARG     8571  10636  13650
2 BRA     6797   7519   8918
3 CHL    13231  16265  16800
4 COL     5334   6131   6624
5 MEX     8347  10046  11091
```

`pivot_wider()`: de largo a ancho

¿Cuándo usar cada uno?

`pivot_longer()`: para pasar de formato **planilla** a formato **tidy** (antes de analizar).
`pivot_wider()`: para pasar de formato **tidy** a formato **tabla** (para presentar o calcular).

pivot_wider(): con prefijo en nombres

```r
pbi_largo |>
  pivot_wider(
    names_from   = anio,
    values_from  = pbi_pc,
    names_prefix = "pbi_"
  )
```

```
# A tibble: 5 x 4
  pais  pbi_2020 pbi_2021 pbi_2022
  <chr>    <dbl>    <dbl>    <dbl>
1 ARG       8571    10636    13650
2 BRA       6797     7519     8918
3 CHL      13231    16265    16800
4 COL       5334     6131     6624
5 MEX       8347    10046    11091
```

`pivot_wider()`: con prefijo en nombres

`names_prefix` agrega un prefijo a los nombres de las nuevas columnas, útil para que los nombres sean más descriptivos.

Caso práctico: calcular crecimiento

Combinando pivots con mutate para calcular el crecimiento del PBI entre 2020 y 2022:

```
pbi_ancho |>
  pivot_longer(starts_with("pbi_"), names_to = "anio",
               names_prefix = "pbi_", values_to = "pbi_pc") |>
  mutate(anio = as.integer(anio)) |>
  filter(anio %in% c(2020, 2022)) |>
  pivot_wider(names_from = anio, values_from = pbi_pc,
              names_prefix = "pbi_") |>
  mutate(crec_pct = round((pbi_2022 / pbi_2020 - 1) * 100, 1)) |>
  arrange(desc(crec_pct))
```

```
# A tibble: 5 x 4
  pais  pbi_2020 pbi_2022 crec_pct
  <chr>    <dbl>    <dbl>    <dbl>
1 ARG       8571    13650     59.3
2 MEX       8347    11091     32.9
3 BRA       6797     8918     31.2
4 CHL      13231    16800     27
5 COL       5334     6624     24.2
```

Parte III: Condicionales

if_else(): condición binaria

if_else() de dplyr es una versión **estricta** de ifelse() de R base:

```
latam |>
  filter(anio == 2022) |>
  mutate(
    nivel_inflacion = if_else(inflacion > 10, "Alta", "Baja")
  ) |>
  select(pais, inflacion, nivel_inflacion)
```

```
# A tibble: 7 x 3
  pais  inflacion nivel_inflacion
  <chr>     <dbl> <chr>
1 ARG        72.4 Alta
2 BRA         5.8 Baja
3 CHL        11.6 Alta
4 COL        10.2 Alta
```

if_else() vs ifelse()

if_else() (dplyr)

- Verifica que `true` y `false` sean del **mismo tipo**
- Más seguro: evita errores silenciosos
- Tiene argumento `missing` para NAs

ifelse() (R base)

- Permite mezclar tipos (puede causar bugs)
- Convierte fechas a numérico sin aviso
- No tiene manejo explícito de NAs

```r
# if_else() da error si los tipos no coinciden:
if_else(TRUE, "texto", 42)    # Error
ifelse(TRUE, "texto", 42)     # Devuelve "texto" (sin avisar)
```

Recomendación

Siempre usar `if_else()` de dplyr dentro de `mutate()`.

case_when(): múltiples condiciones

Cuando hay más de dos categorías, case_when() reemplaza múltiples if_else() anidados:

```
latam |>
  filter(anio == 2022) |>
  mutate(
    categoria_infl = case_when(
      inflacion > 50  ~ "Hiperinflación",
      inflacion > 10  ~ "Alta",
      inflacion > 5   ~ "Moderada",
      .default        = "Baja"
    )
  ) |>
  select(pais, inflacion, categoria_infl)
```

```
# A tibble: 7 x 3
  pais  inflacion categoria_infl
```

case_when(): sintaxis y reglas

La sintaxis es `condición ~ valor`:

```
case_when(
  condicion_1 ~ resultado_1,
  condicion_2 ~ resultado_2,
  condicion_3 ~ resultado_3,
  .default    = resultado_por_defecto
)
```

Reglas importantes

1. Las condiciones se evalúan **en orden**: la primera que se cumple "gana".
2. `.default` captura todo lo que no fue atrapado antes.
3. Todos los resultados deben ser del **mismo tipo** (todos character, todos numeric, etc.).
4. Si no se pone `.default`, los casos no cubiertos quedan como `NA`.

case_when(): ejemplo económico

```r
latam |>
  filter(anio == 2022) |>
  mutate(
    clasificacion_bm = case_when(
      pbi_pc >= 13846 ~ "Ingreso alto",
      pbi_pc >= 4466  ~ "Ingreso medio-alto",
      pbi_pc >= 1136  ~ "Ingreso medio-bajo",
      .default        = "Ingreso bajo"
    )
  ) |>
  select(pais, pbi_pc, clasificacion_bm)
```

```
# A tibble: 7 x 3
  pais  pbi_pc clasificacion_bm
  <chr>  <dbl> <chr>
1 ARG    13650 Ingreso medio-alto
2 BRA     8918 Ingreso medio-alto
3 CHL    16800 Ingreso alto
4 COL     6624 Ingreso medio-alto
5 MEX    11091 Ingreso medio-alto
```

Case When: sintesis

Parte IV: Modelo relacional

¿Qué es un modelo relacional?

En la vida real, los datos casi nunca están en **una sola tabla**. El modelo relacional organiza la información en múltiples tablas vinculadas:

Ejemplo: datos económicos

- Tabla `paises`: código, nombre, continente
- Tabla `indicadores`: código país, año, PBI, inflación
- Tabla `comercio`: exportador, importador, producto, valor

¿Por qué no una sola tabla gigante?

- Evita **redundancia** (repetir datos)
- Reduce **errores** de actualización
- Es más **eficiente** en almacenamiento

Para conectar las tablas necesitamos **claves** (*keys*).

Claves primarias y foráneas

Clave primaria (*primary key*)

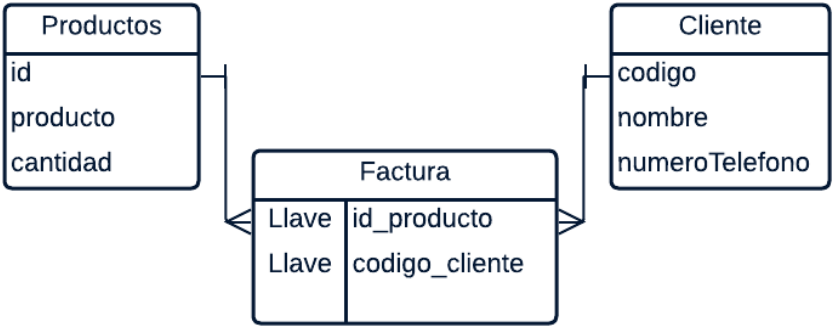
Columna (o combinación de columnas) que **identifica de forma única** cada fila de una tabla. No puede tener valores repetidos ni `NA`.

Clave foránea (*foreign key*)

Columna que **referencia a la clave primaria** de otra tabla. Establece la relación entre tablas.

Tabla	Clave primaria	Clave foránea
`paises`	`codigo_pais`	—
`indicadores`	`codigo_pais` + `anio`	`codigo_pais` → `paises`
`comercio`	`exp` + `imp` + `prod`	`exp`, `imp` → `paises`

Modelo relacional



Verificar una clave primaria en R

Para confirmar que una columna (o combinación) funciona como clave primaria, verificamos que no haya duplicados:

```r
# ¿"pais" es clave primaria de latam?
latam |>
  count(pais) |>
  filter(n > 1)
```

```
# A tibble: 7 x 2
  pais      n
  <chr> <int>
1 ARG       3
2 BRA       3
3 CHL       3
4 COL       3
5 MEX       3
6 PER       3
```

Verificar una clave primaria en R

Para confirmar que una columna (o combinación) funciona como clave primaria, verificamos que no haya duplicados:

```r
# ¿"pais" + "anio" es clave primaria?
latam |>
  count(pais, anio) |>
  filter(n > 1)
```

```
# A tibble: 0 x 3
# i 3 variables: pais <chr>, anio <int>, n <int>
```

```r
# Sin duplicados: pais + anio SÍ es PK
```

Creando tablas para practicar joins

```r
# Tabla de metadatos por país
paises_meta <- tibble(
  codigo = c("ARG","BRA","CHL","COL","MEX","PER","URY","ECU","VEN"),
  nombre = c("Argentina","Brasil","Chile","Colombia","México",
             "Perú","Uruguay","Ecuador","Venezuela"),
  moneda = c("ARS","BRL","CLP","COP","MXN",
             "PEN","UYU","USD","VES")
)

# Tabla de indicadores (solo algunos países)
indicadores_2022 <- latam |>
  filter(anio == 2022) |>
  select(codigo = pais, pbi_pc, inflacion)
```

Observar: `paises_meta` tiene 9 países (incluye Ecuador y Venezuela); `indicadores_2022` tiene solo 7. Esta diferencia será clave para entender los distintos tipos de joins.

Parte V: Joins

¿Qué es un join?

Un **join** combina dos tablas usando una o más columnas en común (las claves):

```
tabla_resultado <- tabla_a |>
  xxx_join(tabla_b, by = "columna_en_comun")
```

Las cuatro variantes principales:

Tipo	Resultado
`left_join()`	Conserva **todas las filas de la tabla izquierda**. Agrega columnas de la derecha donde haya match.
`inner_join()`	Conserva **solo las filas con match** en ambas tablas.
`full_join()`	Conserva **todas las filas de ambas** tablas.
`anti_join()`	Conserva filas de la izquierda **sin match** en la derecha.

left_join(): el más usado

Conserva **todas** las filas de la tabla izquierda y agrega las columnas de la derecha:

```
indicadores_2022 |>
  left_join(paises_meta, by = "codigo")
```

```
# A tibble: 7 x 5
  codigo pbi_pc inflacion nombre    moneda
  <chr>   <dbl>     <dbl> <chr>     <chr>
1 ARG     13650      72.4 Argentina ARS
2 BRA      8918       5.8 Brasil    BRL
3 CHL     16800      11.6 Chile     CLP
4 COL      6624      10.2 Colombia  COP
5 MEX     11091       7.9 México    MXN
6 PER      7126       8.5 Perú      PEN
7 URY     20795       8.3 Uruguay   UYU
```

`left_join()`: el más usado

Los 7 países de `indicadores_2022` se mantienen. Ecuador y Venezuela (que están solo en `paises_meta`) **no aparecen**.

inner_join(): solo los que coinciden

Conserva únicamente las filas que tienen match en **ambas** tablas:

```
paises_meta |>
  inner_join(indicadores_2022, by = "codigo")
```

```
# A tibble: 7 x 5
  codigo nombre    moneda pbi_pc inflacion
  <chr>  <chr>     <chr>   <dbl>     <dbl>
1 ARG    Argentina ARS     13650      72.4
2 BRA    Brasil    BRL      8918       5.8
3 CHL    Chile     CLP     16800      11.6
4 COL    Colombia  COP      6624      10.2
5 MEX    México    MXN     11091       7.9
6 PER    Perú      PEN      7126       8.5
7 URY    Uruguay   UYU     20795       8.3
```

`inner_join()`: solo los que coinciden

Solo aparecen los 7 países que existen en las dos tablas. Ecuador y Venezuela se pierden (no tienen indicadores).

full_join(): todo con todo

Conserva **todas** las filas de ambas tablas, con NA donde no hay match:

```
paises_meta |>
  full_join(indicadores_2022, by = "codigo")
```

```
# A tibble: 9 x 5
  codigo nombre    moneda pbi_pc inflacion
  <chr>  <chr>     <chr>   <dbl>     <dbl>
1 ARG    Argentina ARS     13650      72.4
2 BRA    Brasil    BRL      8918       5.8
3 CHL    Chile     CLP     16800      11.6
4 COL    Colombia  COP      6624      10.2
5 MEX    México    MXN     11091       7.9
6 PER    Perú      PEN      7126       8.5
7 URY    Uruguay   UYU     20795       8.3
8 ECU    Ecuador   USD        NA      NA
```

`full_join()`: todo con todo

Ecuador y Venezuela aparecen con `NA` en `pbi_pc` e `inflacion`. Ninguna fila se pierde.

Todos los joins usuales

a

x1	x2
A	1
B	2
C	3

+

b

x1	x3
A	T
B	F
D	T

=

Mutating Joins

x1	x2	x3
A	1	T
B	2	F
C	3	NA

dplyr::left_join(a, b, by = "x1")

Join matching rows from b to a.

x1	x3	x2
A	T	1
B	F	2
D	T	NA

dplyr::right_join(a, b, by = "x1")

Join matching rows from a to b.

x1	x2	x3
A	1	T
B	2	F

dplyr::inner_join(a, b, by = "x1")

Join data. Retain only rows in both sets.

x1	x2	x3
A	1	T
B	2	F
C	3	NA
D	NA	T

dplyr::full_join(a, b, by = "x1")

Join data. Retain all values, all rows.

anti_join(): ¿qué falta?

Devuelve las filas de la izquierda que **no** tienen match en la derecha:

```
paises_meta |>
  anti_join(indicadores_2022, by = "codigo")
```

```
# A tibble: 2 x 3
  codigo nombre    moneda
  <chr>  <chr>     <chr>
1 ECU    Ecuador   USD
2 VEN    Venezuela VES
```

Muy útil para **diagnóstico**: ¿qué registros de una tabla no se vinculan con la otra?

```
# También se puede invertir:
indicadores_2022 |>
  anti_join(paises_meta, by = "codigo")
```

El argumento `by`: claves con distinto nombre

Si las columnas se llaman diferente en cada tabla, usamos un vector con nombres:

```r
# Renombramos la columna en una tabla para el ejemplo
indicadores_v2 <- indicadores_2022 |>
  rename(cod_pais = codigo)

# Join especificando la equivalencia
paises_meta |>
  left_join(indicadores_v2,
            by = c("codigo" = "cod_pais"))
```

```
# A tibble: 9 x 5
  codigo nombre    moneda pbi_pc inflacion
  <chr>  <chr>     <chr>   <dbl>     <dbl>
1 ARG    Argentina ARS     13650      72.4
2 BRA    Brasil    BRL      8918       5.8
```

Join por múltiples claves

Cuando la clave primaria es una **combinación de columnas**:

```r
# Tabla con calificaciones crediticias
ratings <- tibble(
  pais = c("ARG","ARG","BRA","BRA","CHL","CHL"),
  anio = c(2021, 2022, 2021, 2022, 2021, 2022),
  rating = c("CCC+","CCC-","BB-","BB-","A","A")
)

latam |>
  filter(anio %in% c(2021, 2022), pais %in% c("ARG","BRA","CHL")) |>
  left_join(ratings, by = c("pais", "anio")) |>
  select(pais, anio, pbi_pc, inflacion, rating)
```

```
# A tibble: 6 x 5
  pais   anio pbi_pc inflacion rating
```

¿Cuándo usar cada join?

Join	Caso de uso típico
`left_join()`	Enriquecer una tabla principal con información de una tabla auxiliar. *Ej: agregar nombre de país a datos de comercio.*
`inner_join()`	Quedarse solo con observaciones que existen en ambas fuentes. *Ej: cruzar encuestas de dos períodos, solo individuos que respondieron ambas.*
`full_join()`	Combinar dos fuentes sin perder ningún registro. *Ej: consolidar datos de dos organismos distintos.*
`anti_join()`	Detectar registros sin correspondencia. *Ej: alumnos inscriptos que no rindieron parcial.*

Errores comunes en joins

1. Duplicados inesperados

Si la tabla derecha tiene claves repetidas, el join **multiplica** las filas. Siempre verificar la clave primaria antes de joinear.

2. NAs silenciosos

Un `left_join()` puede generar `NA` en las nuevas columnas sin aviso. Verificar con `anti_join()` cuántas filas no matchearon.

3. Columnas con el mismo nombre

Si ambas tablas tienen columnas con el mismo nombre (además de la clave), R les agrega sufijos `.x` y `.y`. Renombrar antes del join para evitar confusión.

Parte VI: Resumen y cierre

Funciones de hoy

Función	Acción
`group_by() + summarise()`	Estadísticas resumen **por grupo**
`group_by() + mutate()`	Columnas calculadas **relativas al grupo**
`ungroup()`	Eliminar agrupamiento residual
`pivot_longer()`	De formato **ancho** a **largo** (tidy)
`pivot_wider()`	De formato **largo** a **ancho** (tabla)
`if_else()`	Condicional binaria (2 opciones)
`case_when()`	Condicional múltiple (*n* opciones)
`left_join()`	Conservar filas de la **izquierda**
`inner_join()`	Solo filas con **match en ambas**
`full_join()`	Conservar **todas** las filas
`anti_join()`	Filas de la izquierda **sin match**